DESIGN ASSIGNMENT

Design and Development of a Portable Mini Compiler for a Rule-Based Decision Language (RBDL)

Department	Artificial Intelligence and Data Science
Programme	B.Tech
Course Code & Course Name	CSA1403- Compiler Design
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.G.Michael
Assignment Title	Design and Development of a Portable Mini Compiler for a Rule-Based Decision Language (RBDL)
Date of Issue	01/09/2026
Date of Submission	03/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO1, CO2, CO3
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Compiler development, programming-language processing, software tools, and automation of rule-based decision systems

Student Name	ARUNPRASATH R
Register Number	192524077
Section / Batch	2025-2029

B. ASSIGNMENT PROBLEM / CHALLENGE

The assignment focuses on the design and development of a portable mini compiler for a Rule-Based Decision Language (RBDL).

Traditional programming languages require users to write complete programs using general-purpose programming constructs. For simple decision-making applications, this can introduce unnecessary complexity. A specialized rule-based language can provide a simpler way to express conditions and corresponding actions.

The proposed RBDL compiler accepts rule-based statements written in a simple domain-specific language and processes them through the major phases of compilation.

The system is designed to demonstrate how compiler concepts can be applied to a practical language-processing problem. The compiler performs lexical analysis, syntax analysis, semantic validation and rule-oriented processing to produce meaningful results or appropriate error messages.

The challenge is to develop the compiler in a portable and lightweight manner, so that the implementation can be executed in a standard computing environment without requiring a large compiler framework.

C. PROBLEM STATEMENT

Problem / Case / Design Challenge

Rule-based decision systems are commonly used to represent decisions in a structured form such as:

IF temperature > 30

THEN action = "HIGH";

However, directly implementing such rules in a general-purpose programming language requires programming knowledge and additional implementation effort.

The objective of this design assignment is to develop a Portable Mini Compiler for a Rule-Based Decision Language (RBDL) that allows users to express decision rules using a simple and well-defined syntax.

The system should demonstrate the practical application of compiler design concepts while maintaining a simple architecture suitable for educational and small-scale rule-processing applications.

D. REQUIREMENTS AND CONSTRAINTS

The assignment format specifically requires relevant requirements and constraints to be identified, including functional, performance, technical, resource, reliability, security and other applicable constraints.

D.1 Functional Requirements

The proposed RBDL compiler shall provide the following functions:

1. Source Code Input

The system shall accept rule-based programs written using the RBDL syntax.

Example:

RULE temperature_check

IF temperature > 30

THEN decision = "HIGH";

2. Lexical Analysis

The lexical analyzer shall identify:

- Keywords
- Identifiers
- Numeric constants
- String literals
- Operators
- Relational operators
- Assignment operators
- Delimiters

3. Syntax Analysis

The parser shall verify whether the sequence of tokens follows the grammar defined for RBDL.

4. Semantic Validation

The compiler shall perform basic semantic checks such as:

- Valid identifiers.
- Valid operators.
- Valid rule structure.
- Valid assignment statements.
- Appropriate use of values and expressions.

5. Rule Representation

Valid rules shall be converted into an internal representation that can be processed by the rule engine.

6. Rule Evaluation

The system shall evaluate conditions and produce the corresponding decision/action.

7. Error Handling

The compiler shall identify invalid input and provide understandable error messages.

8. Output Generation

For valid programs, the compiler shall display:

- Token information where required.
- Parsing/validation status.
- Generated rule representation.
- Final decision/action.

D.2 Non-Functional Requirements

Portability:The compiler should be capable of running in commonly available environments with minimal modifications.

Usability:The RBDL syntax should be simple enough for users to understand and write basic decision rules.

Reliability:The compiler should consistently produce correct results for valid inputs and meaningful errors for invalid inputs.

Maintainability:The implementation should be modular so that additional language features can be incorporated later.

Performance:The compiler should process small and medium-sized rule sets with reasonable execution time.

Error Reporting:Errors should indicate the general nature and location of the problem wherever possible.

D.3 Technical Requirements

The implementation can use the following tools:

Component	Technology / Tool
Lexical Analysis	Flex / LEX
Syntax Analysis	Bison / YACC
Programming Language	C
Compiler	GCC
Operating System	Windows / Linux
Editor	VS Code / Notepad++
Terminal	Command Prompt / PowerShell
Version Control	Git / GitHub, if required

The supplied assignment format identifies tools such as Python, MATLAB, cloud platforms and Git as examples for computing assignments; the specific tools selected here are appropriate to the compiler-design problem.

D.4 Constraints

1. Language Complexity:The first version of RBDL will support only a limited set of constructs rather than the complete features of a general-purpose programming language.

2. Data Types:The compiler will initially support basic data types such as:

- Integer
- Floating-point values
- Strings
- Boolean conditions

3. Rule Structure:Rules must follow the predefined RBDL grammar.

4. Resource Constraint:The system should operate on a normal personal computer without requiring specialized hardware.

5. Time Constraint:The implementation must be completed within the assignment schedule.

6. Error Handling Constraint:The compiler will provide basic lexical, syntactic and semantic error reporting rather than advanced compiler diagnostics.

7. Portability Constraint:The implementation should avoid unnecessary platform-dependent features.

E. STUDENT WORK

1. PROBLEM UNDERSTANDING AND FORMULATION

1.1 Understanding the Problem

The main problem is to develop a small compiler capable of processing a specialized rule-based language.

A rule-based decision language represents decisions using conditions and corresponding actions. Instead of writing a complete general-purpose program, a user can describe the decision logic using a predefined rule structure.

For example:

RULE eligibility

IF age >= 18

THEN status = "ELIGIBLE";

The compiler must understand this source code and determine whether it is valid according to the language specification.

2. APPLICATION OF COURSE KNOWLEDGE

The assignment requires application of relevant principles, theories, algorithms, design methodologies and compiler concepts rather than merely reproducing theory.

The RBDL compiler applies the following Compiler Design concepts.

2.1 Lexical Analysis

Lexical analysis is the first major phase of the compiler.

The RBDL source program is scanned character by character and grouped into meaningful units called tokens.

The lexical analyzer eliminates unnecessary whitespace and identifies invalid characters.

2.2 Syntax Analysis

The parser receives tokens from the lexical analyzer and verifies whether they follow the RBDL grammar.

For example:

RULE temperature_check

IF temperature > 30

THEN decision = "HIGH";

must follow the predefined rule structure.

An incorrectly ordered statement such as:

RULE temperature_check

THEN decision = "HIGH";

IF temperature > 30

should be rejected if it does not match the grammar.

2.3 Semantic Analysis

Syntax alone does not guarantee that a rule makes logical sense.

Semantic analysis checks conditions such as:

- Whether variables are properly declared/recognized.
- Whether operators are applicable.

2.4 Intermediate Representation

After successful parsing and semantic validation, the rule can be represented internally.

For example:

RULE temperature_check

IF temperature > 30

THEN decision = "HIGH";

can be represented conceptually as:

Rule Name : temperature_check

Variable : temperature

Operator : >

Value   : 30

Action   : decision = "HIGH"

This intermediate representation separates the language-processing stage from rule execution.

2.5 Rule Evaluation

The rule engine evaluates the condition.

For:

temperature > 30

if:

temperature = 35

then:

35 > 30

is TRUE.

Therefore:

decision = "HIGH"

is executed.

If:

temperature = 25

then:

25 > 30

is FALSE, so the associated action is not executed.

2.6 Error Handling

The compiler must handle different categories of errors:

Lexical Error

Invalid character or token.

Syntax Error

Incorrect arrangement of tokens.

Semantic Error

A syntactically valid statement that violates language rules.

Example:

IF temperature > "high"

may produce a semantic/type-related error if the language expects a numeric comparison.

3. PROPOSED SOLUTION / DESIGN / METHODOLOGY

The proposed RBDL compiler uses a modular architecture consisting of several processing stages.

3.1 Module 1 – Lexical Analyzer

The lexical analyzer identifies the basic components of RBDL.

Responsibilities

- Read source characters.
- Remove whitespace.
- Identify keywords.
- Identify identifiers.
- Identify constants.

3.3 Module 2 – Syntax Analyzer

The syntax analyzer verifies the grammatical structure of the source program.

A simplified grammar can be represented as:

program → rule_list

rule_list → rule_list rule

rule → RULE IDENTIFIER IF condition

THEN action ;

condition → expression operator expression

action → IDENTIFIER = value

expression → IDENTIFIER

| NUMBER

| STRING

This grammar can be expanded later to support logical operators and multiple conditions.

3.4 Module 3 – Semantic Analyzer

The semantic analyzer validates the meaning of the parsed rules.

It verifies:

- Identifier validity.
- Data-type compatibility.

- Valid comparisons.
- Valid actions.
- Consistent variable usage.
- Rule consistency.

3.5 Module 4 – Intermediate Rule Representation

The parsed rule is converted into a structured internal format.

Example:

```
Rule {
   name: "temperature_check"
   variable: "temperature"
   operator: ">"
   value: 30
   action_variable: "decision"
   action_value: "HIGH"
}
```

This representation makes rule execution easier.

3.6 Module 5 – Rule Engine

The rule engine evaluates each condition against the supplied input.

Example:

temperature = 35

Rule:

IF temperature > 30

THEN decision = "HIGH";

Evaluation:

35 > 30

TRUE

Output:

Decision = HIGH

4. ALGORITHM

Algorithm: RBDL Compiler

Step 1: Start.

Step 2: Read the RBDL source program.

Step 3: Pass the source program to the lexical analyzer.

Step 4: Generate tokens.

Step 5: Check for lexical errors.

Step 6: If lexical errors exist, display the error and stop processing the invalid statement.

Step 7: Pass valid tokens to the syntax analyzer.

Step 8: Validate the tokens according to the RBDL grammar.

Step 9: If a syntax error occurs, display the error information.

Step 10: Perform semantic validation.

Step 11: Construct the intermediate rule representation.

Step 12: Obtain the required input values.

Step 13: Evaluate the rule condition.

Step 14: If the condition is TRUE, execute the corresponding action.

Step 15: If the condition is FALSE, evaluate the next applicable rule.

Step 16: Display the final decision/output.

Step 17: Stop.

5. PSEUDOCODE

```
BEGIN
READ RBDL source program
CALL lexical_analyzer(source)
IF lexical_error EXISTS
    DISPLAY lexical error
    STOP
END IF
CALL syntax_analyzer(tokens)
```

```
IF syntax_error EXISTS
    DISPLAY syntax error
    STOP
END IF
CALL semantic_analyzer(parse_tree)
IF semantic_error EXISTS
    DISPLAY semantic error
    STOP
END IF
CREATE intermediate_rule_representation
READ input values
FOR each rule
    EVALUATE rule condition
    IF condition is TRUE
        EXECUTE corresponding action
        DISPLAY result
    ELSE
        CONTINUE to next rule
    END IF
```

END FOR

END

6. USE OF MODERN TOOLS

The implementation uses compiler-development tools suitable for constructing a lexical analyzer and parser.

Tool	Purpose
Flex / LEX	Generates the lexical analyzer
Bison / YACC	Generates the syntax parser
GCC	Compiles the generated C source
VS Code / Notepad++	Source-code development
Command Prompt / PowerShell	Build and execution
Git / GitHub	Version control and project management

The assignment format requires appropriate evidence of tool usage and outputs.

7. RESULTS AND VALIDATION

The solution must be validated using appropriate test cases, outputs and screenshots. The assignment format explicitly identifies tables, screenshots and test results as suitable validation evidence.

Test Case 1 – Valid Rule

Input

RULE temperature_check

IF temperature > 30

THEN decision = "HIGH";

Input Value

temperature = 35

Expected Result

Rule condition: TRUE

Decision: HIGH

Actual Result

Rule accepted successfully.

Decision = HIGH

Status: PASS

```
%{
#include "parser.tab.h"
#include <string.h>

int yylineno = 1;
%}

%%

"RULE"          { return RULE; }
"IF"            { return IF; }
"THEN"          { return THEN; }
"ELSE"          { return ELSE; }
">="            { return GE; }
"<="            { return LE; }
"=="            { return EQ; }
"!="            { return NE; }
">"             { return GT; }
"<"             { return LT; }
"="             { return ASSIGN; }
";"             { return SEMICOLON; }
[0-9]+          { yylval.num = atoi(yytext); return NUMBER; }
\"[^"]+\"       { yylval.str = strdup(yytext); return STRING; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.str = strdup(yytext); return IDENTIFIER;
[ \t\r]+        { /* ignore whitespace */ }
'\n             { yylineno++; }
.               { printf("Lexical Error: Invalid character '%s' at line
                        %d\n", yytext, yylineno); }

%%

int yywrap(void) { return 1; }
```

```
C:\Windows\System32\cmd.exe

C:\RBDL>flex lexer.l

C:\RBDL>bison -d parser.y

C:\RBDL>gcc lex.yy.c parser.tab.c -o rbdl.exe

C:\RBDL>rbdl.exe

==========================================
  RBDL Rule-Based Decision Language Compiler
==========================================
Enter RBDL source file name: program1.rbdl
Parsing Successful!
Evaluating Rules...

Enter value for marks: 72
------------------------------------------
Rule: eligibility
Condition: marks >= 50  => TRUE
Decision: status = PASS
------------------------------------------

C:\RBDL>
```

8. ANALYSIS AND ENGINEERING DECISION

The proposed architecture was selected because it clearly separates the different compiler phases.

A monolithic implementation could process the entire RBDL program in a single module, but such an approach would make debugging, maintenance and future language extensions more difficult.

The modular architecture provides the following advantages:

- Clear separation of compiler phases.
- Easier debugging.
- Better maintainability.
- Reusable lexical and parsing components.
- Easier addition of new language features.
- Better demonstration of compiler-design principles.

Alternative Approach

A manually implemented parser could be developed instead of using Bison/YACC.

However, using Flex and Bison/YACC provides a stronger demonstration of formal compiler construction because:

- Flex handles token recognition.
- Bison/YACC handles grammar processing.
- The grammar is explicitly defined.

- Error handling can be integrated with parsing.
- The approach is scalable for additional language constructs.

Therefore, the Flex + Bison/YACC approach is selected as the final implementation strategy.

9. BROADER CONSIDERATIONS

Sustainability:The proposed compiler is software-based and does not require dedicated hardware. It can therefore be developed and executed using existing computing resources.

Accessibility:The simplified RBDL syntax can make basic rule-based decision logic easier to understand for learners who may not be familiar with complex programming languages.

Educational Impact:The project provides practical understanding of:

- Lexical analysis
- Parsing
- Grammar design
- Semantic analysis
- Intermediate representation
- Error handling
- Compiler implementation

Professional Responsibility:The compiler should provide predictable and understandable outputs. Incorrect rule interpretation could result in incorrect decisions, so validation and testing are important.

Security:The implementation should restrict unsupported constructs and validate input before processing it. This reduces unexpected behavior when malformed source programs are supplied.

10. CONCLUSION

The Portable Mini Compiler for a Rule-Based Decision Language (RBDL) provides a practical implementation of fundamental compiler-design concepts.

The proposed system processes RBDL source programs through lexical analysis, syntax analysis, semantic validation, intermediate rule representation and rule evaluation. The modular architecture makes the compiler easier to understand, test, maintain and extend.

The design satisfies the primary requirements of accepting rule-based input, identifying tokens, validating grammar, detecting errors and producing appropriate decisions.

The project also demonstrates how compiler technologies such as LEX/Flex, YACC/Bison and GCC can be combined to create a lightweight domain-specific language processor.

The current implementation is intentionally limited to basic rule constructs. Future versions can support multiple conditions, logical operators, nested rules, functions, variables with richer data types, optimized intermediate representations and more advanced error recovery.

11. STUDENT REFLECTION

This assignment provided practical exposure to the complete process of developing a small language processor rather than learning compiler phases only from theoretical examples.

One of the major learning outcomes was understanding how a source program is gradually transformed from characters into tokens, validated according to grammar, semantically checked and finally processed to produce a meaningful result.

The project also improved understanding of Flex/LEX and Bison/YACC, particularly how lexical rules and grammar specifications work together.

If additional time and resources were available, the RBDL compiler could be extended with more advanced language features such as multiple conditions, logical operators, nested rules, improved semantic analysis, better error recovery and a graphical user interface.

12. REFERENCES

1. A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd Edition, Pearson.
2. Alfred V. Aho and Jeffrey D. Ullman, *Principles of Compiler Design*, Addison-Wesley.
3. Flex documentation and lexical analyzer references.
4. GNU Bison documentation and parser-generation references.
5. GCC documentation.
6. Course materials and lecture notes for Compiler Design.
7. Relevant software documentation and development resources used during implementation.
8. AI-assisted tools used for language clarification, debugging support and documentation preparation, where applicable.